

Video Sunum Metni (Türkçe)

Proje: BLM 4140 — Wi-Fi Başarım Analizi **Hedef süre:** ~10 dakika **Sunum tarzı:** Ekran paylaşımı eşliğinde, sohbet havasında. Slaytlar veya proje dosyaları açıkken anlatım.

Köşeli parantezdeki [1:30] gibi numaralar yaklaşık zamanı gösteriyor; kayıt sırasında saate göz atmanız için.

[0:00] Giriş

Merhaba. Ben Ozan Orhan, öğrenci numaram 21011077. Bu, BLM 4140 Kablosuz ve Mobil Ağlar dersi için Sayın Dr. Mehmet Şükrü Kuran’a sunduğum dönem projem.

Projede şunu inceledim: bir Wi-Fi ağında aynı anda upload yapmaya çalışan cihaz sayısı arttıkça toplam throughput nasıl değişiyor? Bu soruyu NS-3 ağ simülatörü ile inceledim. Esas merak ettiğim şu: G. Bianchi’nin 2000’de gösterdiği “ağ kalabalıklaştıkça throughput düşer” sonucu, 802.11n’in modern MAC özellikleri (A-MPDU, Block-ACK, TXOP) açık olduğunda hâlâ geçerli mi? Ve klasik RTS/CTS el-sıkışması bu durumda gerçekten işe yarıyor mu, yoksa engelliyor mu?

Önümüzdeki on dakikada size ne yaptığımı, simülasyon kodunu ve sonra figürleri ile tabloları tek tek anlatacağım.

[1:00] Arka plan — soru neydi?

2000 yılında G. Bianchi orijinal 802.11 DCF mekanizması üzerine bir makale yayınladı. Matematiksel olarak gösterdi ki çekişen istasyon sayısı arttıkça iki istasyonun aynı anda iletme başlama olasılığı da artar, dolayısıyla çarpışmalar (collision) sıklaşır. Her çarpışma havayı boşa harcar, yani toplam throughput istasyon sayısı arttıkça düşer.

Ama Bianchi’nin analizi 2000’den ve sadece eski DCF’yi kapsıyor. Modern Wi-Fi’de EDCA var (DCF’nin QoS sürümü), TXOP var, birden fazla MPDU’yu tek iletimde gönderen A-MPDU var ve Block-ACK var. Bunların hepsi açıkken Bianchi etkisi belki kaybolur diye düşünebilirsiniz. İşte bu proje tam olarak o soruyu cevaplıyor.

Her senaryoda iki MAC mekanizmasını karşılaştırıyorum:

- Saf EDCA, RTS/CTS yok,
- RTS/CTS açık EDCA — her iletim önce kısa bir kanal-rezerve-et el-sıkışması ile başlıyor.

Ders kitabı sezgisi şöyle: RTS/CTS az istasyon olduğunda zarar verir (ek yük yaratır, faydası yok), çok istasyon olduğunda yarar sağlar (çarpışmaları kısaltır, ucuzlatır). Ben de uplink-yoğun TCP senaryosunda modern MAC özellikleriyle bu sezginin hâlâ geçerli olup olmadığını test etmek istedim.

[2:30] Önemli PHY düzeltmesi

Bir konuyu netleştireyim. Proje açıklama PDF’i “IEEE 802.11ac 2.4 GHz PHY” diyor. Ama 802.11ac yalnızca 5 GHz bandında tanımlı, yani 2.4 GHz’de böyle bir PHY yok. Ders sorumlumuz 14 Mayıs Google Classroom paylaşımında bunu doğrulayıp 802.11n 2.4 GHz PHY’ı 11n hızlarıyla kullanmamızı istedi. Ben de tüm projede 802.11n kullandım. 20 MHz, tek uzaysal akış ve kısa koruma aralığıyla en yüksek MCS, MCS 7 — 72.2 Mbit/s. Tüm yük yüzdelerim bu 72.2 Mbit/s referansına göre.

[3:00] Simülasyon kurulumu — NS-3’te neyi inşa ettim?

Tek bir C++ dosyam var, wifi-project.cc, NS-3’ün scratch/ klasöründe duruyor. Üç komut satırı parametresi alıyor:

- numSTA — 4, 8, 12 veya 20 istasyon,
- macMechanism — ya EDCA ya da RTSCTS,

- `totalLoadPercent` — 72.2 Mbit/s ham hızın %50, %80 veya %90'ı.

Topoloji basit: orijinde tek AP, etrafında 1 metre yarıçaplı bir çember üzerinde STA'lar. Hepsi birbirine çok yakın, hepsi birbirini duyuyor. Gizli düğüm (hidden-node) problemi yok.

Trafik için uplink TCP kullanıyorum. Her STA `TcpSocketFactory` ile bir `OnOff` uygulaması çalıştırıyor; STA başına hız toplam sunulan yükün STA sayısına bölünmüş hali. Yani **toplam** talep istasyon sayısından bağımsız olarak sabit — projenin istediği de bu.

Her senaryo 10 saniye simülasyon trafiği koşturuyor. Her senaryoyu 5 farklı rastgele seed ile 5 kez koşturuyorum. Yani 24 senaryo çarpı 5 seed eşittir 120 simülatör çağrısı. Raporumdaki tüm sayılar bu 5 seed'in ortalaması, hata barları bir standart sapma kadar.

[4:00] Dört MAC özelliğini kodda nasıl açtım?

Kodda dört kritik satırı kısaca göstereyim:

- **EDCA** otomatik. `wifi.SetStandard(WIFI_STANDARD_80211n)` der demez AP ve STA'lar EDCA'yı uygulayan QoS MAC'ini kullanıyor.
- **A-MPDU** için hem STA hem AP MAC'inde `BE_MaxAmpduSize` değerini 65 535 byte yaptım.
- **Block-ACK** A-MPDU açıkken zorunlu, MAC kendi kendine açıyor.
- **TXOP**'u açıkça ayarlamam gereken kısımdı. `AC_BE` için varsayılan TXOP limit sıfır, yani her kanal erişimi sadece tek PPDU taşıyor. Ben `BE_Txop/TxopLimits` üzerinden bunu 3.2 milisaniyeye çıkardım. Böylece AP ve STA'lar bir kanal erişiminde birden fazla birleştirilmiş PPDU patlatabiliyor.
- **RTS/CTS** ise `RtsCtsThreshold` ile kontrol ediliyor. RTS/CTS senaryolarında 100 byte (neredeyse her çerçeve RTS/CTS tetikler), saf EDCA'da 65535 (kapalı).

[5:00] Her şeyi koşturma

120 simülasyonu sırayla koşturmak için `run_all.sh` adında küçük bir bash betiği yazdım. Seed, STA sayısı, MAC mekanizması ve yük üzerinde dönüyor; her simülatör koşusu bir CSV satırını `results_raw.csv`'ye yazıyor. Benim makinemde tüm sweep yaklaşık 15-20 dakika sürüyor — NS-3'ü `optimized profile` ile derlediğim için bu kadar hızlı.

Sonra `plot_results.py` bu CSV'yi okuyup seed'ler arası ortalama alıyor, dört PNG figürü ve üç Mark-down tabloyu üretiyor. Tabloları rapora yapııştırıyorum.

[5:30] Sonuçlar — Şekil 1, %50 yük

Şimdi figürlere tek tek bakalım.

Şekil 1 en hafif yük: ham hızın yüzde ellisi, yani yaklaşık 36 Mbit/s'lik bir toplam talep. Düşük ve orta STA sayılarında her iki eğri 30-34 Mbit/s arasında geziniyor. 36 Mbit/s'e tam ulaşamıyorlar çünkü biraz throughput MAC ek yüküne ve AP'nin geri gönderdiği TCP ACK trafiğine gidiyor.

İlginç olan şey, EDCA ile RTS/CTS eğrilerinin 4, 8 ve 12 STA için neredeyse üst üste binmesi. 4 STA'daki hata barları büyük, yaklaşık 4-5 Mbit/s. Pratikte bu şu demek: düşük yükte iki mekanizmayı ayırt edemezsiniz, hangisini seçtiğiniz çok farketmiyor.

20 STA'ya gelince her iki eğri de sert düşüyor ve RTS/CTS, EDCA'nın biraz üstüne çıkıyor. Yani orta yükte bile 20 istasyonun çekişmesi throughput'u aşındırmaya başlamaya yetiyor.

[6:30] Sonuçlar — Şekil 2, %80 yük

Şekil 2 daha ilginç olan. Yük yüzde 80, yaklaşık 58 Mbit/s talep — ağın doyabileceği seviyenin üstünde. Bu yüzden ölçtüğümüz şey uygulamaların yollamak istediği değil, MAC'in fiilen taşıyabildiği maksimum.

Ve işte sürpriz: RTS/CTS her STA sayısında saf EDCA'yı yeniyor. 4 STA'da fark +8.5 Mbit/s. 8 STA'da +7.9. 12 STA'da +4.7. 20 STA'da +5.7. Bu, "RTS/CTS düşük N'de zarar verir" diyen ders kitabı kuralına aykırı.

Neden böyle oluyor? Çünkü uplink TCP'de her iletim uzun bir A-MPDU. Uzun A-MPDU'lar arasındaki çarpışma çok pahalı — milisaniyeleri boşa harcatıyor. RTS/CTS bunları ucuz kısa RTS çarpışmalarıyla değiştiriyor. Üstüne ikinci bir sebep daha var: AP'nin de istasyonlara TCP ACK göndermesi gerekiyor, ve RTS/CTS AP'nin 4 ile 20 yükleyiciye karşı çekişmeyi kazanmasına yardım ediyor.

Her iki eğri de N arttıkça düşüyor — EDCA 4'ten 20 STA'ya %40 kaybediyor, RTS/CTS %38 — ama RTS/CTS sürekli daha yüksek.

[7:30] Sonuçlar — Şekil 3, %90 yük

Şekil 3 en güçlü senaryo. Talep 65 Mbit/s, ağ tamamen doymuş ve Bianchi-tarzı düşüş zirvede. Saf EDCA 4'ten 20 STA'ya throughput'unun %39.5'ini kaybediyor — neredeyse tam olarak DCF modelinin öngördüğü monoton düşüş eğrisi.

RTS/CTS yine her yerde baskın, ama burada fark N arttıkça büyüyor. 4 STA'da +6.3 Mbit/s ile başlıyor, 20 STA'da +9.65 Mbit/s'e çıkıyor — bu yaklaşık %34'lük bir iyileşme. RTS/CTS aynı zamanda daha yavaş bozuluyor: 4'ten 20 STA'ya sadece %28 kayıp, saf EDCA'da %40. Hata barları dar, 1-4 Mbit/s arası, ve iki eğri açıkça örtüşmüyor.

[8:00] Genel-bakış figürü ve istatistiksel güven

Şekil 4 altı eğriyi tek bir grafiğe koyuyor. EDCA mavi, RTS/CTS kırmızı, çizgi stili yükü gösteriyor. Üç şey hemen göze çarpıyor: hiçbir eğri yukarı çıkmıyor, kırmızı eğriler %80 ve %90 yükte mavinin üzerinde, ve %50 yükte iki eğri neredeyse aynı.

Güvenilirlik konusunda: her veri noktası 5 bağımsız simülasyon koşusunun ortalaması, hata barları örnek standart sapma. Çoğu nokta için stdev 3 Mbit/s'in altında ve RTS/CTS'nin %80 ile %90 yükteki üstünlüğü gürültünün birkaç katı büyüklükte. Daha önce 3 seed ile bir kalibrasyon koşusu da yaptım — ek iki seed eklendiğinde çoğu nokta için ortalama 1 Mbit/s'den az kayıp ve EDCA-RTS/CTS sıralaması hiç değişmedi. Yani 5 seed yeterli.

[9:00] Sonuç — ne öğrendim?

Tüm bunlar bir araya gelince ne çıkıyor?

Birincisi, Bianchi'nin 25 yıllık sonucu hâlâ doğru. 802.11n'in modern MAC özellikleri açık bile olsa, çekişen istasyon sayısı arttıkça throughput düşüyor. Saf EDCA'da %50 yükte %31 kayıp, %80 ve %90 yükte yaklaşık %40 kayıp var.

İkincisi, RTS/CTS yoğun uplink TCP altında şaşırtıcı şekilde işe yarıyor. Klasik öğretide der ki: RTS/CTS düşük istasyon sayılarında zarar verir. Ama bu yargı idealize edilmiş doymuş UDP'ye dayanıyor. Uplink TCP ve modern A-MPDU ile, yük %80'e ulaştığında RTS/CTS her STA sayısında saf EDCA'yı yeniyor. %90 yük ve 20 STA'da üstünlüğü +9.65 Mbit/s'e, yani yaklaşık %34'e ulaşıyor.

Üçüncüsü, düşük yükte hangi MAC'i seçtiğinizin pek bir önemi yok.

Pratik çıkarım: Yoğun, uplink-ağırlıklı bir 802.11n ağda — örneğin 20 öğrencinin aynı anda upload yaptığı bir sınıf — ağ kalabalıklaştığı an RTS/CTS açmak net bir kazanç. Her ders kitabının "ek yük" diye uyarmasına rağmen.

[9:45] Teşekkür

Bu kadardı. İzlediğiniz için teşekkür ederim. Tüm rapor, simülasyon kaynak kodu, ham CSV sonuçları ve dört figür proje ZIP'inde mevcut. Sorularınız olursa lütfen e-posta atın. Teşekkürler.